



Cahier des charges — BedView

Sujet D : Indicateur de gestion des lits hospitaliers

Évaluation — Interopérabilité en santé

Membres du groupe : [Nom 1], [Nom 2], [Nom 3]

Programme : FIE4 — ISIS

Dépôt GitHub : partagé avec LucChattyFyrstain

Serveur FHIR : <https://hapi.fhir.org/baseR4>

Date : 15 septembre 2026

Cahier des charges — BedView

Sujet D : Indicateur de gestion des lits hospitaliers

15 septembre 2026 – Serveur FHIR : <https://hapi.fhir.org/baseR4>

1 Présentation du projet

BedView est une application web permettant de consulter l'état de lits hospitaliers à partir de données récupérées sur le serveur HAPI FHIR. L'application présente les lits dans un tableau de bord, affiche les ressources FHIR utilisées et permet d'exporter les informations au format CSV.

2 Besoin métier

Un responsable de service souhaite connaître les états déclarés des lits de son périmètre et transmettre ces informations à un autre outil de gestion. L'application doit répondre à la question suivante :

« Quels sont les lits de mon périmètre, quels états sont renseignés et quelles informations puis-je exporter ? »

Elle doit permettre de :

- Consulter la liste des lits.
- Distinguer les différents états opérationnels.
- Identifier les informations manquantes ou non reconnues.
- Afficher des compteurs par état.
- Exporter les informations dans un format exploitable par un autre logiciel.

Les indicateurs portent uniquement sur les lits chargés dans l'application. Ils ne représentent pas la capacité réelle d'un établissement.

3 Utilisateur et responsabilités

Responsable de service. Il consulte le tableau de bord, recherche un lit, filtre les résultats, actualise les données et télécharge le CSV.

4 Scénario principal d'utilisation

1. L'utilisateur ouvre BedView.
2. Elle vérifie la connexion au serveur et récupère les lits depuis HAPI.
3. Elle affiche les tableaux, les compteurs (informations en temps réel).
4. L'utilisateur recherche un lit ou filtre les résultats par état.
5. Il sélectionne un lit pour consulter ses informations.
6. Il visualise puis télécharge le CSV des lignes filtrées.
7. Après modification de l'état d'un lit par un tiers, il reçoit une notification/pop up, puis actualise la page pour observer le changement.

5 Périmètre fonctionnel

5.1. Fonctionnalités obligatoires

Réf.	Description
F01	Connexion au serveur — l'application effectue de véritables appels HTTPS à HAPI ; elle ne remplace pas silencieusement une erreur de connexion.
F02	Tableau métier — affiche pour chaque lit son identifiant, son nom, son état opérationnel, la date de dernière modification et les éventuels avertissements.
F03	Indicateurs — affiche le nombre total de lits et les effectifs par état ; les états inconnus apparaissent explicitement.
F04	Recherche et filtres — recherche par nom ou identifiant, filtrage par état.
F05	Inspection FHIR — affiche le JSON exact des ressources reçues, sans le reconstruire à partir du tableau métier.
F06	Correspondance des codes — montre le système et le code source, le code cible, son libellé et le résultat du mapping.
F07	Export CSV — aperçu puis téléchargement des lignes correspondant au filtre appliqué.
F08	Erreurs et traçabilité — présente les erreurs, les avertissements et un journal des appels au serveur.
F09	Actualisation — un bouton relance la lecture des données depuis HAPI.

5.2. Éléments hors périmètre

Cette version ne prend pas en charge :

- L'affectation d'un patient à un lit.
- La consultation de dossiers patients.
- Les réservations de lits.
- Faute de temps, l'authentification hospitalière ne sera pas prise en compte.
- La gestion multi-utilisateur.
- L'historique clinique.
- Le calcul d'un taux réglementaire d'occupation.

L'interface utilisateur fonctionne en lecture seule. La préparation des données fictives est réalisée séparément.

6 Choix des standards et spécifications

6.1. Version FHIR

Le projet utilise FHIR R4, avec la version de référence 4.0.1, correspondant au serveur choisi.

Adresse du serveur : <https://hapi.fhir.org/baseR4>

6.2. Protocole et format d'échange

Les échanges utilisent :

- Une API REST.
- Le protocole HTTPS.
- Le format JSON FHIR.
- L'en-tête HTTP « Accept: application/fhir+json ».

6.3. Ressources utilisées

Ressource	Rôle
Location	Ressource métier principale représentant chaque lit.
Bundle	Conteneur des résultats d'une recherche (réponses « searchset » et pages suivantes).
CapabilityStatement	Description des capacités du serveur, récupérée via <code>GET /metadata</code> .
OperationOutcome	Ressource permettant d'expliquer certaines erreurs retournées par le serveur.
ConceptMap	Ressource locale décrivant les correspondances entre codes source et codes cible.

6.4. Profil et guide d'implémentation

Le projet utilise la ressource de base Location de FHIR R4, accompagnée de contraintes locales explicites. Les contraintes locales sont documentées et contrôlées par l'application. Elles ne constituent pas un profil FHIR formel publié.

6.5. Format cible

Le format cible est le CSV, conformément au sujet D. Le contrat CSV propre à BedView porte la version 1.0.0.

7 Modèle de données

Chaque lit est représenté par une ressource Location.

7.1. Champs utilisés

- **resourceType** — la valeur doit être « Location ».
- **id** — identifiant de la ressource sur le serveur, nécessaire pour identifier un lit et éviter de le compter plusieurs fois.
- **meta.tag** — marqueur permettant de sélectionner les données fictives appartenant au groupe.
- **mode** — la valeur « instance » est utilisée pour les lits préparés ; une ressource « kind » est exclue (décrit un type de lieu) ; si absent, conservée avec avertissement.
- **physicalType.coding** — système `http://terminology.hl7.org/CodeSystem/location-phys` code bd (identifie un lit).
- **name** — nom du lit ; si absent, affichage de l'identifiant et signalement du remplacement.

- **status** — statut administratif : « active », « suspended » ou « inactive » ; valeur absente ou non reconnue signalée.
- **operationalStatus** — état opérationnel du lit (system, version, code, display) ; si non renseigné, état local « INCONNU ».
- **meta.lastUpdated** — date de dernière modification ; si absente, affichage « non renseigné ».

7.2. Règles de conservation

- Un champ facultatif manquant ne provoque pas la suppression d'un lit du tableau.
- Une ressource non reconnue comme lit est exclue des indicateurs. Cette exclusion reste visible dans les informations de qualité.

8 Terminologies et mapping sémantique

8.1. Terminologie source

Les états opérationnels utilisent le système <http://terminology.hl7.org/CodeSystem/v2-0116>. La référence retenue est la version 2.2.0 du CodeSystem, publiée dans HL7 Terminology 4.0.0.

8.2. Terminologie cible

Le projet définit un vocabulaire local :

- Identifiant : urn:bedview:codesystem:etat-lit — Version : 1.0.0

La correspondance est identifiée par :

- Identifiant : urn:bedview:conceptmap:etat-lit — Version : 1.0.0

8.3. Correspondances retenues

Code source (v2-0116)	Code cible local
O — Occupied	OCCUPE — Occupé
U — Unoccupied	INOCCUPE — Inoccupé
H — Housekeeping	ENTRETIEN — Entretien
C — Closed	FERME — Fermé
K — Contaminated	CONTAMINE — Contaminé
I — Isolated	ISOLE — Isolé

Ces correspondances sont définies comme équivalentes dans le périmètre local du projet. Un lit inoccupé n'est pas automatiquement disponible pour accueillir un patient : le code

indique uniquement une absence d'occupation déclarée. L'état « isolé » ne permet pas non plus de déduire si le lit est occupé.

8.4. Règles de transformation

- Le mapping repose sur le couple « système + code ». Le libellé seul ne sert jamais à déterminer la correspondance.
- Si l'état ou le code est absent, la cible devient « INCONNU » avec le résultat « absent ».
- Si le système est absent ou différent du système attendu, la cible devient « INCONNU » avec le résultat « `systeme_non_pris_en_charge` ».
- Si le code n'est pas reconnu, la cible devient « INCONNU » avec le résultat « `code_non_mappe` ».
- Si la version source est absente, les codes connus sont transformés avec un avertissement indiquant que la version n'est pas précisée.
- Si une version source explicite diffère de la version retenue, la cible devient « INCONNU » avec le résultat « `version_non_prise_en_charge` ».
- La version source n'est jamais inventée dans l'export.
- « INCONNU » est un résultat technique du traitement, pas une équivalence avec un concept HL7.

8.5. Mise en œuvre du mapping

Les correspondances sont enregistrées dans un ConceptMap FHIR R4 local et versionné. L'application lit ce ConceptMap pour effectuer la transformation. Les six correspondances connues utilisent la valeur « équivalent ». L'opération distante « \$translate » n'est pas nécessaire pour cette version : le mapping local permet de montrer les règles appliquées sans dépendre d'un service terminologique supplémentaire.

9 Règles de calcul des indicateurs

Le périmètre de calcul contient les ressources sélectionnées par le marqueur du groupe, reconnues comme des lits, et dédoublées par identifiant.

Les règles sont les suivantes :

- Le total correspond au nombre de lits retenus, quel que soit leur statut administratif.
- Chaque lit appartient à une seule catégorie d'état local : OCCUPE, INOCCUPE, ENTRETIEN, FERME, CONTAMINE, ISOLE ou INCONNU.
- La somme des sept catégories doit être égale au nombre de lits retenus.

- Les compteurs et le CSV suivent le filtre appliqué ; le total avant filtrage reste affiché séparément.
- Les ressources exclues et les lits avec avertissements sont comptabilisés.
- Un statut administratif « active » ne signifie pas que le lit est inoccupé.
- Aucun taux d'occupation hospitalier n'est calculé, car la capacité réellement exploitable n'est pas établie.
- Si le chargement est incomplet, les résultats portent la mention « données partielles » et l'export final est désactivé.

10 Interactions avec le serveur HAPI

10.1. Vérification des capacités

GET <https://hapi.fhir.org/baseR4/metadata> — vérifie la version FHIR et les interactions annoncées par le serveur.

10.2. Recherche des lits du groupe

GET https://hapi.fhir.org/baseR4/Location?_tag=urn:bedview:dataset|IDENTIFIANT-GROUPE
L'identifiant du groupe est unique et conservé dans la configuration. Les paramètres sont encodés correctement lors de la construction de l'URL.

10.3. Lecture d'un lit

GET <https://hapi.fhir.org/baseR4/Location/IDENTIFIANT-LIT>

10.4. Pagination

L'application suit les liens « next » fournis dans le Bundle jusqu'à leur absence. Elle ne reconstruit pas les liens de pagination et accepte uniquement les destinations correspondant au serveur configuré. Les doublons sont supprimés à partir de l'identifiant du lit ; si plusieurs versions du même lit apparaissent, un avertissement est produit et la dernière occurrence reçue est conservée.

10.5. Conservation des états absents

La recherche principale ne filtre pas sur la présence d'un état opérationnel, afin de ne pas exclure les lits dont l'état est absent et de ne pas fausser les indicateurs.

11 Préparation des données fictives

Un script séparé prépare dix lits fictifs sur HAPI. Chaque lit comporte :

- Un nom indiquant clairement qu'il s'agit d'une donnée de démonstration.

- Le type physique « bd ».
- Le mode « instance ».
- Le marqueur du groupe.
- Un statut administratif « active ».
- Un état opérationnel, sauf pour le cas de test prévu sans état.

Les ressources sont créées avec `POST /Location`. Le script conserve les identifiants renvoyés dans un manifeste local ; une nouvelle exécution utilise ce manifeste pour éviter les doublons.

Après la préparation, l'application relit les données depuis HAPI. Le fichier de préparation ne fournit jamais directement les données du tableau de bord.

La démonstration de mise à jour utilise un `lit` du manifeste avec `PUT /Location/IDENTIFIANT-LIT` et un contrôle de version avec « If-Match ». Aucune ressource appartenant à un tiers n'est modifiée.

12 Architecture technique

12.1. Technologies retenues

- HTML pour la structure de l'interface.
- CSS pour la présentation.
- JavaScript pour la logique applicative.
- Vite pour le développement local.
- Fetch pour les appels HTTP.
- HAPI FHIR pour le stockage des ressources de démonstration.

L'application ne possède pas de base de données propre. Les versions exactes des dépendances seront fixées lors de la réalisation.

12.2. Organisation des modules

Module	Rôle
Client FHIR	Gère les capacités du serveur, les recherches, la pagination, les lectures et les erreurs HTTP.
Contrôle des données	Identifie les lits, supprime les doublons et produit les avertissements.
Module de mapping	Charge le ConceptMap et transforme les codes source.
Module d'indicateurs	Calcule les effectifs à partir des données transformées et filtrées.
Module CSV	Construit l'aperçu et le fichier exporté à partir des mêmes lignes.
Interface utilisateur	Présente les quatre onglets, les filtres, les résultats et les messages.
Journal des échanges	Conserve les informations utiles sur les appels au serveur.

12.3. Circulation des données

Le navigateur interroge HAPI. Le client FHIR reçoit les ressources JSON. Le module de contrôle sélectionne les lits. Le module de mapping transforme les codes. Les données transformées alimentent le tableau, les indicateurs et le CSV. Les JSON sources restent disponibles pour inspection.

La connexion directe depuis le navigateur sera vérifiée pour les autorisations CORS. Si elle est bloquée, un proxy de développement limité à HAPI sera utilisé.

13 Analyse selon les couches ReEIF

13.1. Infrastructure et sécurité

L'application fonctionne dans un navigateur et communique avec HAPI en HTTPS. Elle prévoit un délai d'expiration des appels et contrôle les destinations utilisées.

13.2. Applications

Les échanges reposent sur l'API REST FHIR R4. Les modules séparent la lecture, la transformation et la présentation. Le format cible est un CSV documenté.

13.3. Information

Le lit constitue l'objet métier principal. Les codes source sont conservés. Le vocabulaire cible et le mapping sont versionnés. Les données absentes et les limites d'interprétation sont explicites.

13.4. Métier et processus

Le responsable consulte les états déclarés et exporte les résultats. L'application ne prend pas de décision d'affectation d'un patient.

13.5. Organisation et gouvernance

Le groupe est responsable de son jeu fictif, des règles de transformation et de leur vérification. Les membres documentent leurs choix et se répartissent les travaux.

13.6. Juridique

Le prototype utilise uniquement des lits fictifs sans données personnelles. Une utilisation hospitalière réelle nécessiterait une analyse juridique et des mécanismes de sécurité adaptés. HAPI étant un serveur public de test, le projet ne suppose pas que les données y seront conservées durablement.

14 Description des écrans

14.1. Onglet « Tableau de bord »

- Le nom du projet et le périmètre sélectionné.
- Un bouton « Actualiser ».
- L'heure de récupération des données.
- Les compteurs par état.
- Une recherche par nom ou identifiant.
- Un filtre par état.
- Le tableau des lits.
- Les avertissements de qualité.

Les couleurs sont accompagnées de libellés textuels.

14.2. Onglet « Ressources FHIR »

- La liste des ressources utilisées.
- La sélection d'un lit.
- Le JSON exact reçu.
- Un lien vers la ressource HAPI.
- Le journal des échanges.

- Les avertissements associés.

Le contenu reçu est affiché comme du texte et n'est pas exécuté comme du HTML.

14.3. Onglet « Conversion CSV »

- Le nombre de lignes à exporter.
- Le filtre appliqué.
- L'aperçu du CSV.
- Le bouton de téléchargement.
- Les explications sur les valeurs manquantes et les transformations.

L'export est désactivé pendant un chargement incomplet ou après un échec d'actualisation.

14.4. Onglet « Correspondances »

- Les six correspondances de référence.
- Les identifiants des systèmes source et cible.
- La version du mapping.
- Le détail de la transformation pour chaque lit.
- Les résultats non mappés et leurs raisons.
- Le ConceptMap local consultable.

15 Spécification du CSV

15.1. Format

- Version du contrat : 1.0.0
- Nom du fichier : `bedview-lits-AAAAMMJJ-HHMMSS.csv`
- Encodage : UTF-8 — Séparateur : virgule — Fin de ligne : CRLF
- En-tête : obligatoire — Granularité : une ligne par lit visible

Une valeur absente est exportée dans une cellule vide. Un état cible non déterminé vaut « INCONNU ».

15.2. Colonnes

Colonne	Contenu
id_lit	Identifiant FHIR.
nom_lit	Nom du lit, ou identifiant si le nom manque.
statut_administratif	Valeur de Location.status.
systeme_source	Système de l'état opérationnel.
version_source	Version reçue, vide si absente.
code_source	Code d'état reçu.
systeme_cible	urn:bedview:codesystem:etat-lit.
code_cible	Résultat du mapping.
libelle_cible	Libellé local.
resultat_mapping	Résultat de la transformation.
avertissements	Messages séparés par un point-virgule.
version_mapping	1.0.0.
derniere_modification	Valeur de meta.lastUpdated.
date_recuperation	Date et heure UTC de début du chargement.
url_source	Adresse complète de la ressource HAPI.

15.3. Règles de sérialisation

- Un champ contenant une virgule, un guillemet ou un retour à la ligne est entouré de guillemets.
- Les guillemets présents dans une valeur sont doublés.
- Pour éviter l'interprétation de textes externes comme des formules de tableur, une apostrophe est ajoutée aux cellules à risque, notamment celles commençant par « = », « + », « - » ou « @ ». Cette protection est signalée.
- Le JSON source reste inchangé.

15.4. Limites et pertes d'information

- Le CSV ne conserve pas tous les champs FHIR, les extensions ou la hiérarchie des lieux.
- Il ne permet pas de reconstruire entièrement la ressource d'origine.
- Le remplacement d'un nom absent par un identifiant est signalé.
- La date de récupération correspond à la lecture des données ; elle ne représente pas la date d'observation de l'état du lit.
- La date « meta.lastUpdated » indique une modification de la ressource, pas nécessairement un changement d'occupation.

- Le CSV constitue un contrat local d'échange et non un document clinique normalisé.

16 Gestion des erreurs et traçabilité

16.1. Comportements attendus

Situation	Comportement attendu
Réseau indisponible / appel >15s	Message d'échec, proposition de réessayer. Pas de données fictives de remplacement.
Erreur HTTP 4xx ou 5xx	Statut affiché et diagnostic OperationOutcome lorsqu'il existe.
JSON invalide / type inattendu	Chargement arrêté, erreur expliquée.
Aucun lit trouvé	État vide explicite, compteurs à zéro.
Champ facultatif absent	Ligne conservée, information manquante signalée.
Code non reconnu	Code source conservé, état local « INCONNU ».
Échec d'une page de résultats	Données marquées partielles, export désactivé.
Échec après chargement réussi	Anciennes données visibles, mention « non actualisées », export désactivé.
Création de données de test refusée	Échec explicite affiché, pas de simulation de succès.

16.2. Journal des échanges

Le journal conserve en mémoire les 100 derniers appels de la session avec : la date et l'heure, la méthode HTTP, l'URL appelée, le statut HTTP, la durée, le nombre de ressources reçues et l'erreur éventuelle.

Les JSON reçus et leurs métadonnées permettent de vérifier les transformations. Ce journal pédagogique ne constitue pas un dispositif d'audit hospitalier.

17 Jeu de démonstration

Le jeu prévu contient dix lits fictifs :

- 3 lits occupés : O.
- 2 lits inoccupés : U.
- 1 lit en entretien : H.
- 1 lit fermé : C.

- 1 lit contaminé : K.
- 1 lit isolé : I.
- 1 lit sans état opérationnel.

Les états renseignés utilisent le système retenu et la version 2.2.0. Tous les lits possèdent un statut administratif « active », afin de montrer que ce statut ne détermine pas leur occupation.

Résultat attendu : 10 lits au total, répartis entre les six états connus et un état inconnu.

18 Critères d'acceptation et recette

Réf.	Critère
T01	Connexion réelle — les appels HAPI sont visibles et les ressources chargées portent le marqueur du groupe.
T02	Comptage — les dix lits sont affichés avec la répartition exacte du jeu de démonstration.
T03	État absent — le lit sans état reste présent avec la valeur « INCONNU ».
T04	Filtrage — le filtre « OCCUPE » affiche trois lignes et produit un CSV de trois lignes.
T05	Inspection de la source — le JSON affiché correspond à la ressource reçue et à l'identifiant sélectionné.
T06	Mapping — système source, code, cible, version et résultat visibles.
T07	CSV avec caractères particuliers — une virgule ou des guillemets dans un nom ne décalent pas les colonnes.
T08	Système différent — un code « O » associé à un autre système ne devient pas automatiquement « OCCUPE ».
T09	Version non prise en charge — une version source différente produit « INCONNU » et un avertissement explicite.
T10	Pagination et doublons — toutes les pages sont parcourues, un lit dupliqué n'est compté qu'une fois.
T11	Erreur réseau — une panne ou un échec de pagination produit un message et bloque l'export final.
T12	Actualisation — après passage d'un lit de « U » vers « O » sur HAPI, le total reste à dix (quatre occupés, un inoccupé).
T13	Ressource hors périmètre — une chambre reçue à la place d'un lit est exclue et l'exclusion est signalée.
T14	Protection CSV — un nom commençant par « = » est neutralisé dans l'export, la transformation est signalée.

Les tests T01 à T06 et T12 sont vérifiés avec HAPI. Les cas limites peuvent utiliser des réponses simulées dans les tests automatisés. Ces simulations ne remplacent pas la

connexion de l'application au serveur.

19 Planning de réalisation

Créneau	Contenu
0 à 20 min	Initialisation du projet, vérification des capacités et de CORS, préparation du jeu fictif.
20 à 50 min	Client FHIR, pagination, contrôle des données et tableau métier.
50 à 75 min	ConceptMap local, transformation des codes, compteurs et filtres.
75 à 95 min	Vue JSON, aperçu CSV et téléchargement.
95 à 110 min	Journal, erreurs et avertissements.
110 à 120 min	Recette principale, documentation et préparation du dépôt.

Ce planning est une estimation. Si l'intégration nécessite davantage de temps, la présentation visuelle sera simplifiée en conservant les fonctionnalités évaluées.

20 État du projet à la fin de la conception

Le cahier des charges définit le périmètre, les données, les interactions, les règles de transformation et les critères de validation.

Le développement, la création du jeu fictif sur HAPI, les tests de recette et le partage du dépôt seront réalisés pendant la phase suivante.